SIMPLIFICATION AS SEARCH IN MAXIMA: THE FULLSIMP PACKAGE AND ITS EXTENSIONS

CHRIS DAWES

Abstract. Maxima offers many transformations that rewrite an expression into an equivalent form, and leaves the choice among them to the user. The `fullsimp` package by E. Majzoub makes that choice by search: it applies a selection of built-in transformations in turn and keeps the smallest expression it meets, accepting bounded increases in size along the way. This paper describes that algorithm and a series of extensions to it. Two corrections remove ways in which the result depended on the names of variables and functions. A numerical equivalence check rejects rewrites that hold on only part of the domain; it draws test points from a pairwise covering design and evaluates them with the conventions Maxima itself uses. A beam search guarantees, whenever the check can evaluate the input, a result no larger than any single verified transformation of it. Splitting sums and products into parts with disjoint variables keeps unrelated parts from interfering, and an ordered list of form preferences lets the user rank candidates before size. We report the effect of each extension on the package's test corpus and on targeted examples, and describe the checks used to validate them.

1. Introduction

A computer algebra system simplifies at two levels. Automatic simplification applies a fixed set of rules to every expression it builds, such as collecting like terms and ordering factors. Beyond that, the user asks for particular transformations, each of which brings an expression into a normal form of its own, such as a quotient of expanded polynomials or a product of irreducible factors [14, 1]. Which of these forms suits a reader depends on the expression and on the purpose [14]; Carette gives a formal definition of the representational part of the question, based on minimum description length [2].

Maxima [13] follows this design. Its simplifier runs on every result, and functions such as `ratsimp`, `factor`, `trigsimp`, `radcan` and `logcontract` provide the further transformations. A user who wants the simplest form of an expression tries several of them, often in sequence, and compares the results by eye. Mathematica automates this step: `FullSimplify` searches over its

Date: 29 October 2026.

2020 *Mathematics Subject Classification.* 68W30.

Key words and phrases. computer algebra, simplification, Maxima, numerical identity testing, covering arrays, beam search.

transformations, guided by a complexity function that the user may replace [18].

The `fullsimp` package by E. Majzoub [11] brings a search of this kind to Maxima. It grew out of code by D. Scherfgen, and Majzoub described it on the Maxima mailing list in December 2025 [10]. It measures an expression by the number of cons cells in its internal representation plus the length of its printed form, applies a list of built-in transformations chosen for the functions present, and returns the smallest result. While it searches, it lets an intermediate expression grow, so that a transformation that makes the expression larger can lead to a smaller one later.

This paper documents that algorithm and seven changes made to it, each developed and tested on its own:

- a correction to how one transformation evaluates its input (Section 4);
- a size measure that does not depend on the names of variables or functions (Section 4.2);
- packaging that installs under either name and keeps the package's own names out of the user's session (Section 4.3);
- a numerical check that rejects candidates whose value differs from the input (Section 5);
- a beam search that keeps the input available to every transformation (Section 6);
- a split of sums and products into parts with disjoint variables (Section 7);
- an ordered list of form preferences that rank candidates before size (Section 8).

Section 2 reviews the parts of Maxima the search relies on, Section 3 describes the original algorithm, Section 9 describes how we validated the changes and measures their combined effect, and Section 10 lists known limitations.

The code is available from the author's fork of the package [12], with one branch per change. Each change has a written specification in the repository's `openspec` directory. The test files are at the top level, the test scripts in `tests`, and the benchmark scripts behind Sections 7 and 9 are `paper/fsimp_bench.mac` and `paper/fsimp_pairs.mac`. We took all measurements with Maxima 5.50.0 on SBCL 2.6.9, on an Apple M4 with macOS 15.6.

2. Background

2.1. **Expressions.** Maxima represents an expression as a Lisp list whose first element names the operator, so that Maxima stores $x + yz$ as

```
((mplus) $x ((mtimes) $y $z)).
```

The simplifier keeps expressions in a canonical internal form. Subtraction and division do not appear in it: $a - b$ is $a + (-1) \cdot b$ and x/y is $x \cdot y^{-1}$. Roots are powers, so $\sqrt{x}$ is $x^{1/2}$. Maxima functions see this form when the option `inflag` is true. The simplifier also orders the terms of sums and the

factors of products by a fixed ordering on their parts, in which the names of variables take part, so renaming a variable can reorder an expression.

A function with no definition stays unevaluated: `f(x)` is an expression in its own right, and the quoted `'f(x)` is its noun form. Built-in mathematical functions such as sin and Γ carry simplification routines, which Maxima records as the `operators` property of the operator.

Maxima's numeric conventions follow its default `domain : real`. An odd root of a negative real is real: the simplifier rewrites $(-8)^{1/3}$ as -2, and evaluating `(-8.0)^(1/3)` gives -2 to within rounding. For other arguments, functions return principal values.

The assume database holds facts declared by `assume` and `declare`, such as $x > 0$ or "n is an integer". Transformations consult it, and `facts()` returns its contents.

Local variables of a `block` are dynamically scoped: a binding made by a block is visible to every function called while the block runs. Maxima functions also evaluate their arguments. Both facts matter in Section 4.

2.2. **Transformations.** Table 1 lists the transformations `fullsimp` uses. We call a transformation *value-preserving* when its result equals its input at every point where both are defined. Three of them rely on rules such as $\sqrt{xy} = \sqrt{x}\sqrt{y}$ or $\log a + \log b = \log ab$, which hold only on part of the complex domain, and the package also counts the Γ routines among those that are not value-preserving. `radcan`, for example, rewrites $\sqrt{x^2 + 2x + 1}$ as $x + 1$, which differs from the input for $x < -1$.

2.3. **Size.** The package measures an expression by

$$\operatorname{size}(e) = \operatorname{conssize}(e) + \operatorname{slength}(e), \tag{1}$$

where $\operatorname{slength}(e)$ is the length of the one-line string Maxima prints for e, and conssize is Maxima's own measure of the internal list, defined by $\operatorname{conssize}(a) = 0$ for an atom a and

$$\operatorname{conssize}(f(a_1, \ldots, a_k)) = 1 + \sum_{i=1}^{k} \bigl(1 + \operatorname{conssize}(a_i)\bigr).$$

For example, $\operatorname{conssize}(\sin x + x^2) = 8$. Maxima's `scsimp` uses conssize as its size measure as well. Other measures exist: Maxima's `charsets` package emulates Maple's `length` [17], and Mathematica's default complexity function ranks forms mainly by leaf count, counting integers with more digits as more complex [18]. As S. Macrakis noted on the Maxima list, size is not the only reasonable objective: $x/(x^2 - 1)$ and $x/((x - 1)(x + 1))$ are close in length, and the second shows the poles [9]. Section 8 returns to this point. In the package the size function is `fscost`, which the user may redefine.

2.4. **Test material.** The package's tests are Maxima *rtest* files: pairs of an input and its expected result. The three original files, by Majzoub, hold 174 calls of `fullsimp` on a single expression; with one regression test added by

TABLE 1. Transformations used by `fullsimp`. Value-preserving transformations agree with their input wherever both are defined; the check of Section 5 treats the others, and any user routine, as not value-preserving.

Transformation	Effect	Value-preserving
`resimplify`, `combine`	resimplify; join terms with a common denominator	yes
`expand`, `multthru`	distribute products over sums	yes
`xthru`, `ratsimp`	combine into one fraction; rational normal form	yes
`factor`	factor over the integers	yes
`minfactorial`	cancel factorials in quotients	yes
`trigsimp`, `trigrat`	trigonometric identities; rational form in sin, cos	yes
`trigexpand`, `trigreduce`	expand multiple angles; reduce powers	yes
`exponentialize`, `demoivre`	trigonometric to exponential and back	yes
`fstrigsimp`	the package's own trigonometric rules (Section 3.2)	yes
`radcan`	canonical form for exponentials and radicals, logarithms included	no
`gamma_simp`	simplify Γ functions (`gamma_simp` package)	no
`factorial_simp`	simplify factorials (`gamma_simp` package)	no
`fs_rationalize`	`ratsimp` with `algebraic : true` (Section 8)	yes
`rootscontract`	$\sqrt{x}\sqrt{y} \to \sqrt{xy}$	no
`logcontract`	$\log a + \log b \to \log ab$	no

the change of Section 4.2 they hold 175, and we call these the *corpus*. Of these, 168 contain only plain variables, which the renaming experiments of Section 4.2 need. The new test files bring the number of `fullsimp` calls in all seven files to 198.

3. The original algorithm

3.1. **Choosing transformations.** `fullsimp` first maps itself over the entries of lists and matrices (and the elements of sets). For a single expression e it builds a list T of transformations from the functions e contains. The base list is

```
resimplify, expand, combine, radcan, ratsimp, rootscontract,
          xthru, multthru, factor, minfactorial.
```

If e contains a trigonometric or hyperbolic function, an exponential or e, the list

```
demoivre, exponentialize, factor, fstrigsimp, trigexpand,
    factor, trigsimp, trigreduce, trigrat, fstrigsimp
```

goes in front. Logarithms add `logcontract`, and Γ functions and factorials add the routines of the `gamma_simp` package. The user can remove entries by naming them in the call, as in `fullsimp(e, radcan)`, and append routines of their own through the list `fs_custom_simp`. After each transformation the package applies `combine`.

3.2. **The trigonometric rules.** `fstrigsimp` applies 31 rewrite rules, in five families: twelve half-angle rules such as $(1+\cos a)/\sin a \to \cot(a/2)$, eight double-angle rules in tan, three rules for $\cos a \pm \mathrm{i}\sin a$, three for csc and cot such as $\tan a + \cot a \to 2\csc 2a$, and five that recognise hyperbolic functions, four of them in exponentials. Most rules carry a pattern variable for a cofactor, so they apply inside a product. The rules run in a fixed order through `apply1`, which rewrites every matching subexpression, and `fstrigsimp` keeps the result of a rule only if it is smaller.

3.3. **The sweep.** The search keeps two expressions: the best found so far, b, and a current state, s. It passes over T twice. Each transformation produces a candidate from s. A candidate no larger than s becomes the new state, and if it is smaller than b it becomes the new best. A larger candidate becomes the state if its relative increase in size is below a threshold $\sigma = 0.8$. The search returns b (Algorithm 1).

Algorithm 1 The sweep

Input: expression e, transformations $T = (t_1, \ldots, t_n)$, threshold $\sigma = 0.8$
Output: b with $\mathrm{size}(b) \le \mathrm{size}(e)$

$s \leftarrow e$; $b \leftarrow e$
for $pass = 1, 2$ **do**
  for $i = 1, \ldots, n$ **do**
    $c \leftarrow \mathrm{combine}(t_i(s))$
    if $\mathrm{size}(c) \le \mathrm{size}(s)$ **then**
      $s \leftarrow c$
      if $\mathrm{size}(c) < \mathrm{size}(b)$ **then** $b \leftarrow c$
    else if $(\mathrm{size}(c) - \mathrm{size}(s))/\mathrm{size}(s) < \sigma$ **then**
      $s \leftarrow c$ ▷ accept a larger state
return b

The sweep calls $2n$ transformations, is deterministic, and never returns an expression larger than its input. The package's documentation describes the search as a simplified form of simulated annealing [7]. Its acceptance rule, which takes any worse state whose deterioration stays below a threshold, is

also the rule of threshold accepting [4], here with a fixed threshold instead of a decreasing one. A typical run on

$$\frac{\cos t + 1}{(\mathrm{i}\sin p - \cos p)\sin t}$$

applies `demoivre` and `exponentialize` without gain, then `fstrigsimp`, whose rules

$$\frac{1+\cos a}{\sin a} \to \cot\frac{a}{2}, \qquad -\cos a + \mathrm{i}\sin a \to -\mathrm{e}^{-\mathrm{i}a}$$

give $-\mathrm{e}^{\mathrm{i}p}\cot(t/2)$, reducing the size from 68 to 39.

4. Dependence on the user's names

Three problems concerned the user's names. Two made results depend on the names in the input expression, and the third let the package's own names interfere with the user's. We found them while developing the later changes, and corrected each in a change of its own, placed before the others.

4.1. **Double evaluation in fstrigsimp.** `fstrigsimp` applied its i-th rule to its argument e by `apply('apply1, [e, fstrgrule[i]])`. `apply` evaluates its list argument, which turns the symbol e into the expression, and `apply1` then evaluates that expression again. So Maxima evaluated every variable of the user's expression in the dynamic environment of the search, and a variable named like a local variable of any function on the call stack took that variable's value. Table 2 shows the effect on the example of the previous section, written in different variable names.

Table 2. Results for $(\cos t + 1)/((\mathrm{i}\sin v - \cos v)\sin t)$ before the correction, by the name of v. The expected result is $-\mathrm{e}^{\mathrm{i}v}\cot(t/2)$.

v	Role on the call stack	Result
`i`	loop index in `fstrigsimp` (value 4 when the rule fires)	$-\mathrm{e}^{4\mathrm{i}}\cot(t/2)$
`L`	pass counter in the sweep (value 1)	$-\mathrm{e}^{\mathrm{i}}\cot(t/2)$
`Li`	size of the argument in `fstrigsimp` (value 70)	$-\mathrm{e}^{70\mathrm{i}}\cot(t/2)$
`r`, `e`	the sweep's transformation list; the argument itself	input unchanged

The correction passes the quoted symbol,

```
apply('apply1, ['e, fstrgrule[i]]),
```

so that `apply1` evaluates the symbol once, to the expression. A regression test binds a variable named `p` outside `fullsimp` and checks that the result does not change. A more general harness, described in Section 9, does the same for every local name on the call path.

4.2. **Names in the cost.** The string term in (1) counts each occurrence of a variable by the length of its name, so the size of an expression, and with it the path of the sweep, depends on the names of the variables. On the example above the second transformation, `exponentialize`, raises the size by about 80 per cent. With v named `zzz` the size goes from 72 to 132, an increase of 0.833, which the threshold rejects. With `zzzz` it goes from 74 to 133, an increase of 0.797, which the threshold accepts. From that state the later transformations do not reach the cot form, and the sweep returns its input. The names of four, five, six and eight characters we tried all behaved like `zzzz`.

The correction discounts user-chosen names. Call a function *unknown* if its operator has no `operators` property and is not a subscripted variable; these are the calls, quoted or not, to functions the user has not defined. Let

$$\text{excess}(e) = \sum_v n_v(e)\,(|v| - 1) + \sum_f m_f(e)\,(|f| - 1),$$

where v ranges over the variables of e (as Maxima's `listofvars` returns them, including subscripted variables such as c_1), $n_v(e)$ is the number of occurrences of v, f ranges over unknown functions, $m_f(e)$ is the number of calls to f, and $|\cdot|$ is the printed length. The new measure is

$$\text{size}(e) = \text{conssize}(e) + \text{slength}(e) - \text{excess}(e). \tag{2}$$

Each occurrence prints exactly once, so (2) counts every user name as one character. conssize never depended on names. Renaming may still reorder terms, but the count does not depend on their order.

To measure the effect, we renamed every variable of the 168 corpus inputs whose variables are plain symbols (Section 2.4), appending eight characters to each name. The old measure changed for all 168 expressions and the new measure for none. Simplifying the renamed inputs and renaming the results back gave a result different from the original in one case under the old measure, and in none under the new one. A further 17 synthetic cases with unknown functions, renamed from `f` to `fffffffff` and so on, gave one difference under the old measure and none under the new.

4.3. **Names in the user's session.** Loading a Maxima package runs its files in the user's session, so every name they define becomes the user's name too. `fullsimp` declared a, b and z as pattern variables for its trigonometric rules, so that afterwards a rule the user wrote with `defrule(r, a + x, 0)` matched $b + x$ as well. Its 31 rules were named `tr001` to `tr405`, and its Lisp file defined a user-level `conssize`. All of these now carry an `fs` prefix. Maxima's own `trigsimp` declares a in the same way when it first loads; the package loads it at once and removes that declaration, unless a already had one, since the rules of `trigsimp` are compiled by then.

Two more effects reached the user. Loading the HTML help index emptied Maxima's own index, from 3164 entries to 9, because the generated file called a function that clears the table first; it now adds its entries. The

search printed a line from `rat` for every routine that met a float, and the check printed Maxima's error message for every test point where the input had no value, 48 lines for one call with an unevaluated integral. Both are now silent. The file `fsimp.mac` also loads its companion files from its own directory, so a manual install works without the `docs` directory on the search path, and a small `fullsimp.mac` makes `load("fullsimp")` work, which the package manager `mxpm` needs. After these changes every function, variable and rule the package defines starts with `fs`, apart from `fullsimp`. An input with Γ or a factorial still loads Maxima's `gamma_simp` package, whose own functions the package cannot rename; excluding its routines keeps it out.

5. Checking candidates numerically

5.1. **Motivation.** A search that keeps the smallest candidate favours the transformations of Table 1 that are not value-preserving, since their results tend to be small. To measure how often this happens we checked the expected results of the three original test files numerically against their inputs, with an evaluator independent of the package (Section 9). Of the 174 expected results, 11 were not equal to their input at all points. They included

$$\log a + \log b \to \log ab, \qquad \sqrt{x}\,\sqrt{y} \to \sqrt{xy},$$
$$\sqrt{x^2+2x+1} \to x+1, \qquad \log\frac{x+1}{x^2-1} \to -\log(x-1),$$

each of which holds for some values of the variables and fails for others. The check described in this section compares every candidate with the input at a fixed set of points and discards any candidate that disagrees at one. Assumptions in force restrict the points, so that

```
assume(x > -1)$ fullsimp(sqrt(x^2+2*x+1));
```

returns $x+1$, while the same call without the assumption returns its input.

5.2. **Test points.** Numerical testing of polynomial identities at random points has well-known error bounds [15, 19]. The expressions here contain logarithms, roots and inverse trigonometric functions, whose branch cuts make an identity hold on a region rather than almost everywhere. Whether a point detects a failure then depends on where each variable lies relative to the cuts. $\log a - \log b \to \log(a/b)$ fails exactly when $\arg a - \arg b$ falls outside $(-\pi, \pi]$, which needs a and b in suitable different regions; when an earlier version of our audit sampled all variables of a point from the same region, it found 9 of the 11 failures, and only mixed signs found the other two.

Each variable of a test point therefore takes a value of a given *kind*: positive real (`pos`), negative real (`neg`), or complex in quadrant j (`q`j). A complex value has modulus m and argument $(j-1)\pi/2 + (\pi/2)(0.15 + 0.7u)$ with u uniform in $[0, 1)$, which keeps it clear of the axes. The modulus is uniform in $[0.2, 1)$ or in $[1, 2)$.

Covering every combination of kinds over n variables would take 4^n points for four kinds, 1024 for five variables. Failures such as the logarithm rule

above need two variables in particular regions, so we require only that every *pair* of variables meets every pair of kinds: for any two variables and any two kinds, some test point gives the first variable the first kind and the second variable the second. A set of points with this property is a pairwise covering array [3].

For up to five variables an orthogonal array $\mathrm{OA}(16, 5, 4, 2)$ gives this coverage with 16 points. It is a table of 16 rows and 5 columns with entries in $\{0, 1, 2, 3\}$, in which any two columns, read together down the rows, show each of the 16 ordered pairs of symbols exactly once [5]. Each row is a test point, each column a variable, and each entry the kind that variable takes (Table 3). The rows correspond to the pairs (a, b) with $a, b \in \{0, 1, 2, 3\}$, and the columns are

$$a, \quad b, \quad a + b, \quad 2a + b, \quad 3a + b,$$

computed in the field $\mathbb{F}_4$ with four elements, written $0, 1, 2, 3$. Addition in $\mathbb{F}_4$ is the bitwise exclusive or of the two-bit numbers, so $1 + 1 = 0$ and $2 + 1 = 3$. Multiplication by 0 and 1 is as usual, and otherwise $2 \cdot 2 = 3$, $2 \cdot 3 = 1$ and $3 \cdot 3 = 2$. The row $(a, b) = (1, 1)$, for example, reads $1, 1, 0, 3, 2$.

To see that any two columns show every pair exactly once, take the columns $\lambda a + b$ and $\mu a + b$ with $\lambda \neq \mu$, and ask which row gives them the values (u, v). Subtracting the two equations gives $(\lambda - \mu)a = u - v$, so $a = (u - v)/(\lambda - \mu)$ and then $b = u - \lambda a$: exactly one row. The column b is the case $\lambda = 0$, and pairs involving the first column, a, work the same way. The division needs a field. With arithmetic modulo 4, where 2 has no inverse, the columns $a + b$ and $3a + b$ differ by $2a$, which is always even, so pairs whose values differ by an odd number would never appear.

TABLE 3. The orthogonal array $\mathrm{OA}(16, 5, 4, 2)$, transposed: column i is test point i, row j the kind symbol of variable j.

Point	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$v_1 = a$	0	0	0	0	1	1	1	1	2	2	2	2	3	3	3	3
$v_2 = b$	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3
$v_3 = a + b$	0	1	2	3	1	0	3	2	2	3	0	1	3	2	1	0
$v_4 = 2a + b$	0	1	2	3	2	3	0	1	3	2	1	0	1	0	3	2
$v_5 = 3a + b$	0	1	2	3	3	2	1	0	1	0	3	2	2	3	0	1

With 16 points no such table has more than five columns [5]. Beyond five variables, the package fills each further column point by point with the kind that covers the most pairs with earlier columns not yet covered, so the number of points stays the same and the coverage of later pairs becomes partial. Variables that occur inside a function with a branch cut, or in a power with a non-integer exponent, take the first columns, so up to five of them have full pairwise coverage.

The 48 test points use the rows three times:

Points	Kinds for symbols $0, 1, 2, 3$	Modulus
1–16	`pos`, `neg`, `q2`, `q3`	$[0.2, 1)$
17–32	`pos`, `neg`, `q2`, `q3`	$[1, 2)$
33–48	`pos`, `neg`, `q1`, `q4`	alternating by variable

The first two passes concentrate on the left half-plane, where the principal cuts of the logarithm and of fractional powers lie; the third makes sure all four quadrants appear. A random draw replaces any point at which the input has no value, up to 120 draws in all. Each check seeds the random state with a fixed value and restores it afterwards, so results are reproducible and the user's random sequence does not change.

5.3. **Assumptions.** The check reads `facts()`. A variable declared real or integer, or appearing in an inequality, gets a real or integer domain; facts of the form $v > c$, $v \geq c$, $v < c$ or $v \leq c$ with constant c give bounds, and the check rounds integer bounds inward. A real variable without bounds takes the sign of its kind, so that sign combinations still appear. Other facts among the variables, such as $x > y$, act as filters that a point must satisfy. If the check cannot draw half the points under the assumptions, the check falls back to unrestricted complex points and reports this at `fsdebug >= 1`.

5.4. **Evaluation.** The first version of the check let Maxima evaluate each point, with `float` and `rectform`. On the expanded form of $(a+b+c+d+u)^{10}$, which has about a thousand terms, `fullsimp` then took 3.9 s instead of about 0.16 s, because Maxima carries out complex arithmetic symbolically. Applied to a whole quotient p/q, `rectform` also computed its argument as $\arg p - \arg q$, which can fall outside $(-\pi, \pi]$.

The check therefore uses a small evaluator written in Common Lisp. It walks the expression bottom-up in double-precision complex arithmetic, so every function receives a number. Sums, products, powers, exp, log and the trigonometric and hyperbolic functions map to their Common Lisp counterparts, whose branch cuts follow the principal-value conventions [16]. Maxima evaluates any other function, such as Γ or a Bessel function, at the numeric arguments. A result whose imaginary part is a signed zero becomes real, so that two equal expressions do not fall on opposite sides of a cut through the sign of zero [6]. A negative real raised to a rational power with an odd denominator takes the real root, as in Maxima. With the time for the large example back at 0.22 s, against 0.16 s unchecked, checking costs about a third there (Table 8).

A candidate c agrees with the input e at a point if $|c - e| \leq 10^{-8}(1 + |e|)$. When a candidate disagrees in double precision, Maxima's own bottom-up evaluation recomputes the input and the candidate with 40-digit bigfloats, and the check rejects the candidate only if the disagreement remains.

We added the real-root rule after a cross-check of the two evaluations. With principal roots throughout, the Lisp evaluator agreed with $\exp(\frac{1}{3}\log x) \to x^{1/3}$ at every point, and the check accepted the rewrite, which Maxima itself

evaluates differently: at $x = -8$ the input is $1 + \sqrt{3}\,\mathrm{i}$ and the result is -2. Over a grid of 30 functions at 15 points, 450 pairs in all and including points on the cuts, the two evaluations disagreed on 16 pairs before the rule and on 7 after it, all on the imaginary axis, which the test points avoid.

5.5. **Unknown functions.** An expression such as $f(\sin^2 x + \cos^2 x)$ has no numeric value. Before evaluation the check replaces each unknown function f of n arguments (in the sense of Section 4.2) by a fixed stand-in

$$\varphi_h(a_1, \ldots, a_n) = \exp\Big(\sum_{k=1}^{n} \gamma_k a_k\Big) + \sum_{k=1}^{n} \delta_k a_k^3 + \tfrac{1}{2},$$

with rational coefficients $\gamma_k \in [0.3, 0.8)$ and $\delta_k \in [0.2, 0.7)$ derived from a hash h of the name of f. An identity that holds for every f holds for φ_h. Different names usually get different stand-ins, so $f(x) - g(x)$ does not vanish; two names whose hashes coincide share one, which would only matter if a transformation exchanged the two functions. With this substitution $f(\sin^2 x + \cos^2 x) + g((x^2 - 1)/(x - 1))$ simplifies to $f(1) + g(x + 1)$.

5.6. **Inputs without a numeric value.** If fewer than three points evaluate, as for an expression containing an unevaluated integral, the check cannot compare values. It then accepts a candidate only if every transformation on the path from the input is value-preserving in the sense of Table 1. Routines added through `fs_custom_simp` never count as value-preserving.

5.7. **What the check does not establish.** The check samples finitely many points. It cannot see a difference confined to a set of measure zero, so $(x^2 - 1)/(x - 1) \to x + 1$ passes, as it does elsewhere in Maxima. Pairwise coverage does not ensure that every combination of three kinds appears, and beyond five branch variables not even every pair. A wrong rewrite that agrees with the input at all 48 points would pass.

6. Beam search

6.1. **Motivation.** In the sweep each transformation sees only the current state. For the derivative

$$\frac{d}{dx} \arctan \frac{x}{\sqrt{1 - x^2}} = \frac{(1 - x^2)^{-1/2} + x^2 (1 - x^2)^{-3/2}}{x^2/(1 - x^2) + 1},$$

`factor` applied to the input gives $1/\sqrt{1 - x^2}$, of size 28. By the time the sweep reaches `factor`, earlier transformations have changed the state, and the sweep returns $1/(\sqrt{1 - x}\,\sqrt{x + 1})$, of size 47. Without the check it returned $\mathrm{i}/\sqrt{x^2 - 1}$, of size 29, which differs from the input for $|x| > 1$.

6.2. **Algorithm.** The beam search [8] keeps a pool of candidates instead of one state (Algorithm 2). The pool always contains the input, the *anchor*, so every transformation sees the original expression. For a width $w \geq 3$ its other members are the $w - 2$ best candidates found so far and one *uphill* candidate: the best-ranked candidate not already in the pool whose size is at most $1 + \sigma$ times that of the best, which keeps the threshold idea of the sweep. The default width is $w = 4$. Each round applies every transformation to every pool member not expanded before. A hash table of printed forms skips candidates already seen. The search stops when a round adds no new member to the pool, or after four rounds. Ties in size go to the candidate found first, so results are reproducible. With $w = 2$ the pool holds the anchor and the best candidate, and with $w = 1$ only the best candidate, so the search climbs greedily.

Algorithm 2 Beam search

Input: expression e, transformations T, width w, round limit R

$C \leftarrow \{e\}$; $P \leftarrow \{e\}$; $X \leftarrow \emptyset$ ▷ candidates, pool, expanded
for *round* $= 1, \ldots, R$ **do**
  for $p \in P \setminus X$ **do**
    $X \leftarrow X \cup \{p\}$
    for $t \in T$ **do**
      $c \leftarrow \text{combine}(t(p))$
      if c not seen before and c passes the check **then** $C \leftarrow C \cup \{c\}$
  $P \leftarrow \{e\} \cup \{\text{the } w - 2 \text{ best of } C\} \cup \{\text{one uphill candidate}\}$ ▷ $w \geq 3$
  if $P \subseteq X$ **then break**
return the best element of C

Proposition 6.1. *Suppose the input has a value at three or more test points, or the check is off. Then the beam search returns an expression no larger than* combine$(t(e))$ *for every $t \in T$ whose result passes the check.*

Proof. The first round expands the anchor e with every $t \in T$. A result $c = \text{combine}(t(e))$ that passes the check either enters C or matches an expression the search met earlier in this round. In the second case the earlier expression is identical, and since the check depends only on the expression, it passed too and is in C. The search returns the best element of C. □

When the input has no value at enough points, acceptance depends on the path. The check rejects a candidate first reached by a transformation that is not value-preserving, and the search marks it as seen, so it skips a later value-preserving path to the same expression. The proposition can then fail; the sweep does not have this property either.

6.3. **Cost.** On the 198 `fullsimp` calls of all seven test files, the beam search was never larger than a verified single transformation, while the sweep was

larger on the derivative above. On the corpus of Section 9 it changed one result, to a different form of equal size: $\cot x \sin 2x$ now gives $\cos 2x + 1$ instead of $2\cos^2 x$, both of size 18. The corpus (Section 2.4) took 2.1 s instead of 1.2 s. The sweep remains available through `fs_search : 'sweep`.

7. Splitting independent parts

7.1. **Motivation.** `fullsimp` chooses one transformation list for the whole expression and applies every transformation to all of it. Consider

$$e = p + \sin^2 x + \cos^2 x, \qquad p = \text{the expanded form of } (a+b+c+d+u)^{10}.$$

The trigonometric term puts the trigonometric transformations in the list. `trigsimp` turns the last two terms into 1, after which `factor` and the other transformations work on $p+1$, a polynomial of degree 10 in five variables with about a thousand terms. Both searches returned an expanded factorisation of size 20149, the sweep in 8.7 s and the beam search in 16.2 s. Simplifying the two parts separately gives $(a+b+c+d+u)^{10} + 1$.

7.2. **Partition.** For a sum or product in internal form, `fullsimp` partitions the terms (or factors) by their variables: two terms belong to the same group if they share a variable, directly or through other terms. This is the finest partition in which no two groups share a variable. The package sets terms without variables aside and splits only if at least two groups contain variables. It simplifies each group by a recursive call, with a transformation list chosen for that group. A polynomial group then sees no trigonometric transformations.

7.3. **Constants.** Constants interact with the groups: $1 - \sin^2 x$ is $\cos^2 x$, and $2\tan x/(1+\tan^2 x)$ is $\sin 2x$. Trying the constants alone and with each group in turn, as a first version did, still missed identities that need a constant in two groups: $1 - \sin^2 x + \tan^2 y + 1$ gave $\cos^2 x + \tan^2 y + 1$, where the unsplit search finds $\cos^2 x + \sec^2 y$, and in $2\cos^2 u - \sin^2 x$, where the simplifier has already cancelled a -1 against a $+1$, there was no constant left to place.

Let c be the sum of the constant terms. Each group g_i either keeps its own result or takes a *shift* k_i, which means the package simplifies $g_i + k_i$ in its place; the combination then adds back $c - \sum_i k_i$. Every group may take $k_i = c$. A group that contains a trigonometric or hyperbolic function may also take $k_i = \pm 1$, which covers the identities with a constant term among those functions, such as $\sin^2 + \cos^2 = 1$, $1 + \tan^2 = \sec^2$, $\cosh^2 - \sinh^2 = 1$ and $2\cos^2 a - 1 = \cos 2a$. The package forms every combination of shifts, and the best wins, ties going to the first. The first combination has no shifts, and the next ones give c to one group in turn, which is the behaviour of the first version. A group whose own result is a constant, such as $\sin^2 y + \cos^2 y$, joins the constants first, so $x^2 + 2x + \sin^2 y + \cos^2 y$ gives $(x+1)^2$. In a product the shifts are factors, and a group may take the constant factor.

Each shift repeats a group's simplification. For the expanded polynomial p above that repetition is the expensive `factor` call splitting is meant to

avoid: $p + \sin^2 x - 1$ takes 3.0 s when p may take the -1. A group whose size exceeds 1000 therefore takes no shift, and the same input takes 0.5 s.

7.4. **Merging.**

Lemma 7.1. *Let X and Y be disjoint sets of variables, f a function of X and g a function of Y, defined on a product of domains. If $f + g$ is constant, then f and g are both constant.*

Proof. Fix $Y = Y_0$. Then $f(X) = h - g(Y_0)$ for all X, where h is the constant value of $f + g$, so f is constant, and then so is g. □

So groups cannot cancel each other beyond constants. More generally, if the whole expression $f + g$ equals another form $F + G$ with F and G in the same variables as f and g, the lemma applied to $(f - F) + (g - G)$ shows that $F - f$ is constant: the forms of the groups can differ from the best whole form only by constants, which is what the shifts provide. Products follow in the same way through quotients, where the parts do not vanish. They can merge, though: $\log a + \log b$ becomes $\log ab$, and under assumptions $\sqrt{x}\sqrt{y}$ becomes $\sqrt{xy}$, across groups. After combining the group results, the package runs one more search restricted to `logcontract` and `rootscontract`, which stay fast on large expressions. Without this step 6 existing tests failed. The package checks the combined result against the whole input, and if the check fails it simplifies the input unsplit.

7.5. **Effect and limits.** The example above now returns $(a+b+c+d+u)^{10}+1$ in 0.44 s, and no result of the corpus changes. With the shifts, $1 - \sin^2 x + \tan^2 y + 1$ gives $\cos^2 x + \sec^2 y$, and $\cosh^2 z - \sin^2 x$ gives $\sinh^2 z + \cos^2 x$, which the unsplit search does not find. To compare the two paths beyond the corpus, we formed 60 random sums and products of pairs of corpus inputs, renaming the variables of the second so that the two share none. Splitting gave a smaller result in 6 of them and a larger one in none. They took 2.5 s with splitting and 1.5 s without it, most of the difference being the shifts. On small inputs the extra simplifications show in Table 8: splitting doubles or triples the time for the two small sums.

The partition can still lose results that need the whole expression. A factorisation across groups, such as $(x + 1)^2 - y^2 = (x + 1 - y)(x + 1 + y)$, is out of reach; under the size measure the factored form is larger and neither path returns it, but it matters with the preferences of Section 8. The package does not divide constants other than ± 1 between groups, such as $\frac{1}{2}$ for each. The size limit only sees an expression as written, so a short expression that is expensive to simplify, such as $(a + b + c + d + u)^{10}$, still takes the constants once. Splitting is on by default and controlled by `fs_split`.

8. Form preferences

8.1. **Ranking.** The smallest form is not always the one wanted. A user may want a denominator without radicals even if the numerator grows, or

exponentials in place of trigonometric functions. The option `fs_prefer` holds an ordered list of preferences $\pi_1, \ldots, \pi_k$. Each preference counts its *violations* $v_i(e)$, and candidates are ranked by

$$\mathrm{rank}(e) = \big(v_1(e), \ldots, v_k(e), \mathrm{size}(e)\big)$$

in lexicographic order. A candidate with fewer violations of an earlier preference ranks better whatever its size. With the default `fs_prefer : []` the rank is $(\mathrm{size}(e))$ and every comparison is the size comparison of the previous sections, so results do not change.

A prototype used a weighted sum, $\mathrm{size}(e) + W \sum v_i(e)$, which we did not adopt. Any fixed W loses to size on a large enough expression, and the threshold rules of both searches compare ratios of sizes, which a large W distorts.

8.2. **Preferences.** Table 4 lists the preferences implemented. Counts work on the internal form, where a denominator is a power u^n with n a negative number. The three preferences about the shape of the expression stop at function arguments; the others count everywhere. `rational_denominator` adds one transformation, `ratsimp` with `algebraic : true`, which rationalises denominators; the others rely on transformations already selected for the expressions they apply to. Pairs that cannot both hold, `factored` with `expanded` and `exponential` with `trigonometric`, raise an error.

TABLE 4. Form preferences and what each counts as a violation.

Preference	Violation
`rational_denominator`	a denominator that is, or contains, a fractional power
`single_fraction`	a sum with a term that has a denominator; a denominator containing a denominator
`factored`	a sum with variables that `factor` rewrites as a product or power
`expanded`	a product with a sum as a factor; a sum raised to an integer power above 1
`[avoid, f, ...]`	a call to one of the named functions
`sin_cos`	a call to tan, cot, sec or csc
`exponential`	a call to a trigonometric or hyperbolic function
`trigonometric`	a power of e whose exponent contains i

8.3. **Interaction with the search.** Both searches choose the best candidate, order the pool and accept steps by rank. The threshold rules use size alone: a step may raise the number of violations, because reaching a preferred form can pass through a less preferred one, as when a polynomial needs expanding before it factors differently. The returned result is always the best-ranked candidate seen, so it ranks no worse than the input, and Proposition 6.1 holds with rank in place of size.

Violation counts add over the groups of Section 7, so splitting respects the ranking, with these adjustments. With `factored` active, the package keeps sums whole, so that factorisations across groups stay reachable, and with `expanded` active it keeps products whole, so that $(x+1)(y+1)$ can expand. With `single_fraction` active, the merging step also tries `xthru`, so that fractions in separate groups combine.

8.4. **Examples.** Table 5 shows results with a single preference active. On the 175 inputs of the corpus, with each preference on its own and in both searches, no result ranked worse than its input, and between 0 (`trigonometric`) and 86 (`exponential`) inputs ended with fewer violations. With `factored` active, the expanded $(a+b+c+d+u)^{10}$ took 0.33 s against 0.30 s without it, measured in the same session.

TABLE 5. Results with one preference active.

Preference	Input	Result
none	$\tan x + \cot x$	$2\csc 2x$
`sin_cos`	$\tan x + \cot x$	$2/\sin 2x$
`rational_denominator`	$x/(\sqrt{x}+1)$	$(x^{3/2}-x)/(x-1)$
`rational_denominator`	$1/(1+\sqrt{2})$	$\sqrt{2}-1$
`single_fraction`	$1/x + 1/y$	$(y+x)/(xy)$
`factored`	$(x+1)^2 - y^2$	$-(y-x-1)(y+x+1)$
`expanded`	$(x+1)^2$	x^2+2x+1
`expanded`	$(x+1)(y+1)$	$xy+y+x+1$
`exponential`	$\cosh x$	$(\mathrm{e}^x + \mathrm{e}^{-x})/2$
`trigonometric`	$\mathrm{e}^{\mathrm{i}x}$	$\mathrm{i}\sin x + \cos x$

The check still applies: with `rational_denominator` active, the check stops $1/\sqrt{x^2+2x+1}$ from becoming $1/(x+1)$. Maxima's simplifier turns $\sqrt{3}/3$ back into $1/\sqrt{3}$, so no search can remove a single radical from a denominator.

9. VALIDATION

9.1. **Tests.** Each change added Maxima regression tests, one per scenario in its written specification. The package's test files now hold 278 tests: the original three files with 15, 112 and 68 tests, and new files for the check (24), the beam search (12), splitting (21) and preferences (26). The packaging change added three to the first file and one to the check's. The documentation's examples run as a further 12 tests. Several further checks run over the inputs of these files.

Audit of expected results: A separate bottom-up evaluator compares every expected result with its input numerically, applying `rectform` and `float` at each node, with 12 points in each of four modes: all positive, all negative, mixed signs and complex. The audit deliberately does not use the package's check. It found the 11 failures

of Section 5, and after the change it reports none among the 195 of the 198 calls of Section 2.4 that have a numeric value.

Single transformations: The script applies each transformation, followed by `combine`, to each input, and compares the results that pass the check with the result of `fullsimp`, testing Proposition 6.1.

Name collisions: For each of 115 names (every local variable and parameter on the call path, and common single letters) and four expressions in that name, the harness compares `fullsimp` with the name unbound and with it bound to a sentinel value in a block around the call, in both searches, with and without preferences. All 1840 runs agree.

Renaming: The script renames variables and unknown functions to long names as in Section 4.2.

Duplicated computations: Where two parts of the code compute the same quantity, we compared their results. This found four faults, all now corrected, among them the evaluator's principal roots (Section 5) and a size computed inline by the beam search without the name discount of (2).

9.2. **Combined effect.** Tables 6 to 8 show the effect of the changes in four configurations. C0 is the original sweep, without the check and without splitting, but with the corrections of Section 4. C1 adds the check, C2 replaces the sweep by the beam search, and C3 adds splitting, which is the default. The corpus of Section 2.4 runs without assumptions in force.

TABLE 6. The corpus of 175 inputs, of total size 9471, in four configurations. Non-identities are results that differ from their input by the audit's evaluator. One run each.

Configuration	Time (s)	Total size of results	Non-identities
C0: sweep, no check	0.69	1869	11
C1: sweep with check	1.23	2099	0
C2: beam search with check	2.13	2099	0
C3: C2 with splitting	1.95	2099	0

From C0 to C1 exactly the 11 results that were not identities change, and the total size rises by 230. From C1 to C2 one result changes form, at equal size, and from C2 to C3 none changes. The corpus consists of the package's own tests, whose expected results come from the sweep; the changes of Sections 6 and 7 show on the examples of Tables 7 and 8.

TABLE 7. Size of the result; p is the expanded $(a+b+c+d+u)^{10}$, and an asterisk marks a result that differs from its input.

Input	Size	C0	C1	C2	C3
$\frac{d}{dx}\arctan(x/\sqrt{1-x^2})$	115	29*	47	28	28
$\tan x + \cot x$	20	18	18	18	18
$\sqrt{x^2+2x+1}$	31	6*	31	31	31
$\log a + \log b$	20	13*	20	20	20
$\frac{1+\cos t}{\sin t} + \frac{e^y - e^{-y}}{e^y + e^{-y}}$	92	52	52	29	29
p	31055	23	23	23	23
$p + \sin^2 x + \cos^2 x$	31085	20149	20149	20149	28

TABLE 8. Time, median of three warm runs.

Input	C0	C1	C2	C3
$\frac{d}{dx}\arctan(x/\sqrt{1-x^2})$	1.3 ms	4.5 ms	16 ms	11 ms
$\tan x + \cot x$	4.0 ms	9.3 ms	16 ms	12 ms
$\sqrt{x^2+2x+1}$	0.9 ms	2.9 ms	2.0 ms	1.9 ms
$\log a + \log b$	1.2 ms	4.0 ms	2.9 ms	9.1 ms
$\frac{1+\cos t}{\sin t} + \frac{e^y - e^{-y}}{e^y + e^{-y}}$	11 ms	20 ms	30 ms	60 ms
p	0.16 s	0.22 s	0.31 s	0.27 s
$p + \sin^2 x + \cos^2 x$	8.7 s	8.7 s	16.2 s	0.44 s

10. LIMITATIONS AND FURTHER WORK

The limits of the check are those listed in Section 5. The two evaluations behind it still disagree on the imaginary axis, where Maxima's floating-point and bigfloat arctangents take opposite sides of the cut; test points avoid the axis, but an argument such as ix with real x reaches it. For inputs without a numeric value, Proposition 6.1 can fail, as Section 6 describes. `fstrigsimp` decides internally which rules to keep by size alone, so preferences and a user's own size function affect how its result ranks but not which rules it applies. The package keeps the sweep for reproducing earlier results; its threshold compares sizes alone, so preferences do not steer its larger steps.

Several directions remain open. A list of further preferences, among them partial fractions, Horner form, collection in a variable, and rectangular or polar complex forms, accompanies the code. Simplification modulo side relations such as $x^2 + y^2 = 1$ could use Maxima's `scsimp` and `ratsubst` as transformations and the relations as constraints on test points. When the check rejects a smaller candidate, the region of test points where it failed could go back to the user as a suggested assumption, such as $x > -1$. A benchmark corpus drawn from outside the package's own tests would show how the changes behave on expressions they were not developed against.

Use of AI tools

Claude Opus 5.5 (Anthropic), working in Claude Code, produced the changes described in Sections 4 to 8, their tests, specifications and documentation, the measurements and the first draft of this paper. The author chose which problems to address, decided between the design options presented, raised several of the problems the paper reports, and approved each change before it went into a commit. A second instance of the same model reviewed the paper, including its references, and its findings, with a check of the references against publisher and library records, led to corrections in the code and the text.

Acknowledgements

The search described in Section 3 and the trigonometric rules are the work of E. Majzoub, building on code by D. Scherfgen. The author thanks the Maxima developers and the contributors to the Maxima-discuss mailing list, whose discussions informed several of the choices described here.

References

[1] B. Buchberger and R. Loos, *Algebraic simplification*, Computer Algebra: Symbolic and Algebraic Computation (B. Buchberger, G. E. Collins, and R. Loos, eds., in cooperation with R. Albrecht), Computing Supplementum, vol. 4, Springer, Vienna, 1982, pp. 11–43.

[2] J. Carette, *Understanding expression simplification*, Proceedings of the 2004 International Symposium on Symbolic and Algebraic Computation (ISSAC), ACM, 2004, pp. 72–79.

[3] D. M. Cohen, S. R. Dalal, M. L. Fredman, and G. C. Patton, *The AETG system: an approach to testing based on combinatorial design*, IEEE Transactions on Software Engineering **23** (1997), no. 7, 437–444.

[4] G. Dueck and T. Scheuer, *Threshold accepting: a general purpose optimization algorithm appearing superior to simulated annealing*, Journal of Computational Physics **90** (1990), no. 1, 161–175.

[5] A. S. Hedayat, N. J. A. Sloane, and J. Stufken, *Orthogonal arrays: theory and applications*, Springer Series in Statistics, Springer, New York, 1999.

[6] W. Kahan, *Branch cuts for complex elementary functions, or much ado about nothing's sign bit*, The State of the Art in Numerical Analysis (A. Iserles and M. J. D. Powell, eds.), Clarendon Press, Oxford, 1987, pp. 165–211.

[7] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, *Optimization by simulated annealing*, Science **220** (1983), no. 4598, 671–680.

[8] B. T. Lowerre, *The HARPY speech recognition system*, Ph.D. thesis, Carnegie Mellon University, 1976.

[9] S. Macrakis, *Re: What am I missing about the Simplify options?*, message to the Maxima-discuss mailing list, 25 May 2018.

[10] E. Majzoub, *Re: defrule/apply1 that hangs Maxima*, message to the Maxima-discuss mailing list, 14 December 2025.

[11] E. Majzoub, *fsimp-maxima*, Maxima package, `https://github.com/QMeqGR/fsimp-maxima`.

[12] C. Dawes, *fsimp-maxima*, fork of [11] with the changes described here, `https://github.com/cmsd2/fsimp-maxima`.

[13] Maxima, a computer algebra system, version 5.50.0, `https://maxima.sourceforge.io`.
[14] J. Moses, *Algebraic simplification: a guide for the perplexed*, Communications of the ACM **14** (1971), no. 8, 527–537.
[15] J. T. Schwartz, *Fast probabilistic algorithms for verification of polynomial identities*, Journal of the ACM **27** (1980), no. 4, 701–717.
[16] G. L. Steele Jr., *Common Lisp: the language*, 2nd ed., Digital Press, 1990, Section 12.5.3.
[17] M. Talon, *Re: defrule/apply1 that hangs Maxima*, message to the Maxima-discuss mailing list, 15 December 2025.
[18] Wolfram Research, *FullSimplify*, Wolfram Language documentation, `https://reference.wolfram.com/language/ref/FullSimplify.html`.
[19] R. Zippel, *Probabilistic algorithms for sparse polynomials*, Symbolic and Algebraic Computation (EUROSAM '79) (E. W. Ng, ed.), Lecture Notes in Computer Science, vol. 72, Springer, 1979, pp. 216–226.

Email address: `cmsd2@cantab.net`